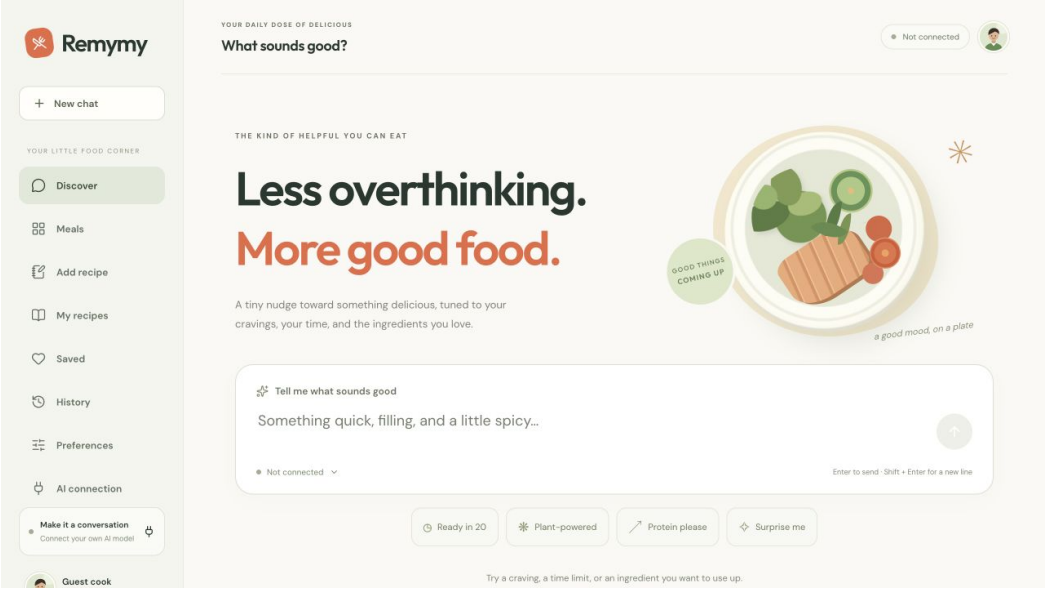
Remymy

User guide

From Docker startup to your first meal conversation and evaluation report.

Local development edition | 30 October 2026

This guide is for a person running the project on their own computer. It covers the current Docker Compose stack and the browser interface included in this repository. Commands use Docker Compose v2 and a Bash-compatible terminal.



Remymy Discover screen

Follow this route

Page	Task
2	Prepare Docker, settings, datasets and models
3	Build and start the services
4-5	Seed and verify the shared data
6-9	Set preferences, rate, save, add recipes and chat
10-12	Capture recommendations, run the judge and inspect scores
13	Check behavior metrics and the offline model benchmark
14-15	Troubleshoot, stop safely and verify completion

Working directory: unless a step says otherwise, run commands from `SystemCode/remymy`. Browser URLs assume the default ports. Do not paste the `$` terminal prompt into commands.

1. Prepare your computer and files

Step 1 - Check Docker

Install Docker Desktop if necessary, launch it, and wait until the engine is running. On Linux, start your Docker Engine service instead. Open a terminal and run:

```
docker info
docker compose version
cd /path/to/practice-project-2/SystemCode/remymy
```

Replace `/path/to/practice-project-2` with your checkout. Use a local port configuration that is free on your machine. The first build and model downloads need internet access, disk space and enough Docker memory for the app, ML services and Langfuse together. If containers are killed, increase Docker's resources and inspect the logs.

Step 2 - Prepare the local settings

Keep an existing `.env.nonprod`. If it is absent, create it in `SystemCode/remymy` with a text editor and add this starter configuration. The file is ignored by Git.

```
ENV=nonprod
NODE_ENV=production
DATA_DIR=./data
APP_PORT=3000
API_PORT=8002
MCP_PORT=8000
MODEL_PORT=8001
LANGFUSE_PORT=3030
MINIO_PORT=9090
MODEL_FILE=hybrid-foodcom-bpr.npz
```

Step 3 - Obtain the project data assets

Large datasets and some weights are not included in Git. Obtain the prepared assets from the project maintainers, or use the source ingestion route on page 4. Keep the filenames below relative to `DATA_DIR`.

File or folder	Needed for
`remymy-food.db`	Easiest first-run seed of the shared recipe database
`raw/food-com/RAW_recipes.csv`	MCP chat recipe catalog
`models/hybrid-foodcom-bpr.npz`	Hybrid ranking service; must match `MODEL_FILE`
`price_mapped_nutrients.csv`	Priced catalog and the API recommender; optional for basic browsing
`foodcom_recipes.csv`	Optional enrichment with fully priced Food.com recipes

The API's reranker and classifier weights live in its source tree. Similar-recipe clustering additionally requires `api/src/remymy_api/inference/weights/recipe_clustering_foodcom/recipe_clustering.joblib`; see `api/README.md` for its artifact source and checksum. Missing optional assets can leave individual ML features unavailable while browsing still works.

2. Start the Docker services

Step 4 - Build and start the full stack

Once the MCP catalog and model artifact on page 2 are present, run:

```
docker compose --env-file .env.nonprod --profile recommender \
  up --build -d --wait --wait-timeout 900
```

Keep Docker Desktop running. `-d` leaves the stack running after the command finishes. The first run builds images and prepares infrastructure, so it takes longer than later runs. If you only need recipe browsing and have not obtained the MCP/model assets, omit `--profile recommender`.

Step 5 - Check startup results

```
docker compose --env-file .env.nonprod --profile recommender ps --all
docker compose --env-file .env.nonprod logs --tail 60 api-migrate api
```

Expect the long-running services to be running, with health checks healthy where configured. `api-migrate`, `models-vocab` and `langfuse-minio-init` are one-time jobs: **Exited (0)** is their successful final state. A nonzero exit or an unhealthy service needs investigation.

Open in your browser	Purpose
`http://localhost:3000`	Remymy web app
`http://localhost:8002/docs`	Data API Swagger interface
`http://localhost:3030`	Local Langfuse traces and scores
`http://localhost:8000/health`	MCP health; full-stack profile only
`http://localhost:8001/health`	Ranking model health; full-stack profile only

Alternative - Use the startup helper

```
./start-all.sh --no-install
```

The helper builds and starts the stack, chooses a free port when a default is busy, opens the app and follows logs. `--no-install` skips host Python environments; containers install their own dependencies. It enables the recommender profile only if both required files exist. Read its printed URLs if a port changes.

Helper lifecycle: it stops existing Remymy containers before starting, then keeps running in the terminal. Pressing Ctrl+C stops the stack and removes containers while keeping volumes. Use the detached Compose command above when you want services to remain running independently of that terminal.

3. Seed the shared recipe data

Step 6 - Let the migration job load the database

Seeding is automatic at startup; there is no separate seed button in the web app. The `api-migrate` job follows this order:

1. If `/state/remymy.db` already contains recipes, validate it and reuse it.

2. Otherwise, if `/data/remymy-food.db` exists, merge its recipe and FoodData Central tables into the state database.

3. Otherwise, require both `RECIPES_SOURCE` and `FDC_SOURCE_DIR` and ingest those source files before merging them.

`/data` is the read-only container mount of `DATA_DIR`. `/state/remymy.db` is the live database on the `remymy-user-data` Docker volume. The merge preserves existing user records; a normal restart does not reset users, ratings or saves.

Prepared database route

Place the provided `remymy-food.db` at `data/remymy-food.db`, then use the startup command on page 3. Do not copy it over the live `/state/remymy.db` file. The migration job handles the import.

Source ingestion route for an unseeded volume

If no prepared database is available, place the Open Recipes source and an extracted FoodData Central Foundation CSV release under `data`. For example:

```
data/13k-recipes.db
data/FoodData_Central_foundation_food_csv_2026-04-30/
```

Add the following to `.env.nonprod`, adjusting paths to match your files:

```
RECIPES_SOURCE=/data/13k-recipes.db
FDC_SOURCE_DIR=/data/FoodData_Central_foundation_food_csv_2026-04-30
FDC_RELEASE=2026-04
```

These are **container paths**, so they start with `/data`, not a host path such as `/Users/...`. The FDC directory should contain the extracted release CSVs, including `food.csv`, `foundation_food.csv`, `food_nutrient.csv`, `nutrient.csv`, `food_portion.csv`, `measure_unit.csv`, `food_category.csv` and `input_food.csv`.

Run the startup command again. The bootstrap creates a temporary ingest, validates it and merges it into the state database. Missing sources cause the migration job to fail and block API startup. Supplying new source paths does not refresh a database that already has recipes; bootstrap intentionally reuses that database.

4. Verify seeding and catalog readiness

Step 7 - Validate the live recipe database

After the API is running, perform a validation that does not import more data:

```
docker compose --env-file .env.nonprod exec -T api \
  python -m migrations.bootstrap --validate-only
curl -fsS http://localhost:8002/ready
curl -fsS 'http://localhost:8002/api/recipes?limit=2&offset=0'
```

Expect the bootstrap to report that the database holds the recipes, `/ready` to return `{"status":"ok"}`, and the list response to contain a positive `total` and recipe `items`. The current prepared local database displayed 13,496 browsable recipes during verification; your source and any added recipes can change that count.

Step 8 - Understand which data powers each feature

Data path	What it powers and when it loads
Shared SQLite recipes	Meals search, recipe details, user recipes, ratings and saves
Priced catalog CSV	On API startup, adds previously unseen `fdc_id` rows to the live catalog
Optional priced Food.com CSV	Adds fully priced recipe entries when the API recommender loads
Raw Food.com CSV + NPZ model	MCP meal recommendations for connected Discover chat

The catalog CSV must be a real file, not a directory. Existing catalog rows are not updated merely by editing the CSV. Browse-only data seeding does not train a model, populate the priced catalog, or create a sample user.

Step 9 - Allow ML features to finish loading

```
docker compose --env-file .env.nonprod logs --tail 100 api
curl -fsS http://localhost:8000/health
curl -fsS http://localhost:8001/health
```

The API becomes ready before its background models finish loading. On first use, term extraction and encoder resources may need downloading. A healthy `/ready` confirms recipe data, not every ML feature. Follow the API logs if tag suggestions, clustering or recommendations return 503.

Keep the data: recipe and user state lives on the user-data volume; ranking snapshots live on the MCP-data volume; Langfuse uses its own persistent volumes. A normal `docker compose down` keeps them. Removing volumes resets those records.

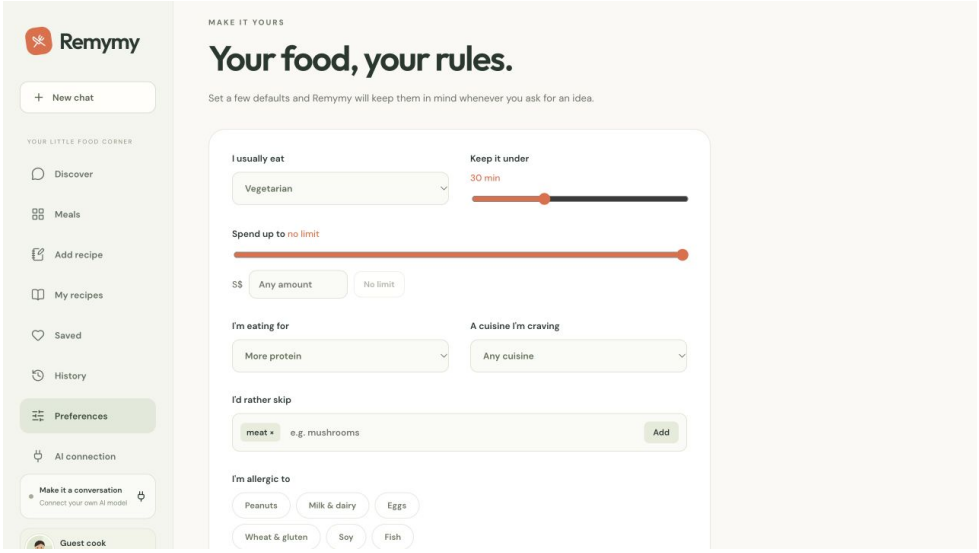
5. Open the app and set preferences

Step 10 - Start in the same browser

Open `http://localhost:3000`. The app creates a guest user automatically and keeps the guest identifier in that browser. No pre-seeded demo account or signup is required. Use the same browser and origin to return to the same guest; clearing cookies or switching browser profiles can create a new guest even while the old records remain in the database.

Step 11 - Save your food preferences

1. Choose **Preferences** in the sidebar.
2. Set **I usually eat** to Anything, Vegetarian, Vegan or the screened Halal option.
3. Adjust **Keep it under** and **Spend up to**. The budget is SGD per meal; use **No limit** if you do not want a budget cap.
4. Select your goal and cuisine. Add ingredient exclusions under **I'd rather skip** and select applicable allergy chips.
5. Click **Save preferences**. **Reset changes** returns the form to the previously saved values.



Preferences screen

Interpret the information shown: the Halal setting screens vegan-tagged recipes and listed ingredients; it does not certify meals as halal. Source recipe records may omit information. Check ingredients and available evidence when choosing a meal. Prices and nutrition are shown only where supported or estimated; a source-backed browsing record can have no nutrition, time or price fields.

6. Browse, rate and save recipes

Step 12 - Find a recipe in Meals

1. Select **Meals**. Confirm the recipe count and cards have loaded.

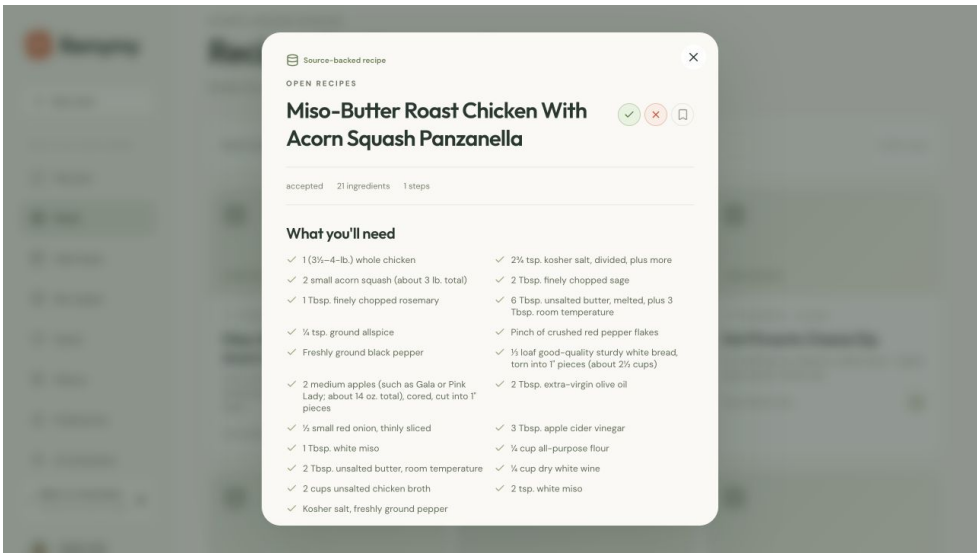
2. Enter a title or ingredient, for example `soup`, in **Search recipes**.

3. Use **Next** and **Previous** at the bottom for more results.

4. Open a card or its **View** button. Read **What you'll need** and **How to make it**.

Step 13 - Seed your own preference history

Use the tick (**Liked it**) or cross (**Not for me**) in the recipe detail view. Rate several different recipes that reflect your actual preferences. The rating appears in **History**, newest first. Rating the same recipe again updates that recipe's entry instead of adding a new distinct item.



Recipe details with rating and bookmark controls

Step 14 - Keep a shortlist

Use the bookmark (**Save for later**) in a recipe detail view. Open **Saved** to find the recipe again. Click its bookmark again to remove the save. **Saved** is your shortlist; **History** records likes and dislikes. A save and a rating are separate actions.

Personalization checkpoint: the Data API recommender uses a cold-start selection through five distinct rated items, then uses its history-based model after more than five. This requires the priced catalog and loaded API model. The current Meals view remains a source-record browser. Connected Discover uses the separate MCP/hybrid ranker; do not assume browser likes retrain that model or automatically become its collaborative user history.

7. Add and revisit your own recipes

Step 15 - Create a recipe

1. Choose **Add recipe** in the sidebar.

2. Enter a recipe name, for example `Tomato chickpea bowl`.

3. Enter each ingredient on its own line. A title and at least one ingredient are required.

4. Add preparation steps, one per line, if desired.

5. Expand **Nutrients per 100 g** only if you have those values. Use non-negative numbers and the displayed units; these fields are optional.

6. Pick tags yourself or click **Suggest tags**. The classifier reads the name and nutrients. Its first request can take longer while models load.

7. Review suggestions. Tap a tag to keep or remove it. Add a custom tag with the text field and **Add**.

8. Click **Save recipe** and wait for the saved confirmation.

Example to type into the form

```
Recipe name:
Tomato chickpea bowl

Ingredients:
1 cup cooked chickpeas
1 tomato, chopped
1 tablespoon olive oil

Steps:
Rinse and drain the chickpeas.
Combine the ingredients in a bowl.
```

Step 16 - Verify the saved result

Select **My recipes**, open the new recipe, and check its title, ingredients, steps and tags. You can open it through the recipe browser, rate it and bookmark it like other stored recipes. My recipes lists the current guest's own creations.

If the classifier is unavailable, choose tags manually and save. A missing clustering artifact may make **Similar recipes** unavailable in a detail view; it does not stop ordinary recipe viewing or saving.

Data boundary: a recipe you add is stored in the shared API database. It is not automatically inserted into the MCP's mounted raw Food.com CSV or trained into the NPZ model. Use **Clear the form** to discard an unfinished form; **New chat** controls conversations, not recipes.

8. Connect AI and use Discover

Step 17 - Connect a provider

1. Select **AI connection**, or click **Not connected** in Discover.

2. Choose a supported **Provider**. Entries marked unavailable cannot be connected from this server.

3. Search for and select a **Model** offered by that provider.

4. Enter your own provider **API key**, then click **Connect & test**.

5. Wait for the successful connection message. Return using **Back to chat** or **Discover**.

A provider connection is required for chat. Browsing recipes, saving, rating and adding your own recipes do not need a chat provider. Provider requests can incur charges. Enter keys yourself; do not include them in screenshots, reports or Git.

Step 18 - Ask for a meal idea

Enter a specific request and press Enter or the send button. Shift+Enter adds a new line. For example:

```
Suggest six vegetarian dinners under 30 minutes.
Avoid mushrooms and explain why each meal fits.
```

Watch **Agent flow** for interpretation, model turns, the MCP recommendation call and answer composition. Open a returned meal for its ingredients, method, estimated nutrition and timing. Ask a follow-up such as `Make these vegan` or `Use chickpeas instead` to refine the conversation.

Step 19 - Recover or start again

If a request fails, read the message, check the provider connection and service health, then use **Try again**. Use **Stop** to interrupt an active response and **New chat** for a fresh conversation. New chat does not erase your saved recipes or ratings.

The full connected flow is: browser message and preferences, app agent, MCP `recommend_meals`, Food.com filtering, hybrid ranking, and the final answer. Missing `mcp` or `models` services prevent that recommendation path even if Meals browsing works.

Connection persistence: the server implementation stores provider keys encrypted on the app-state volume and sets an eight-hour session expiry. Sessions can survive an app restart while valid; **Disconnect** removes the session. The connection screen contains older memory-only wording, so do not rely on restart as a way to disconnect.

Saving a chat suggestion: the current chat meal popup shows details but has no bookmark or rating controls. Use a matching stored recipe in Meals when available, or the documented API with the canonical stored ID. A Food.com chat suggestion may not exist in the smaller imported browsing dataset.

9. Enable and capture ranking evaluations

Step 20 - Configure capture in the MCP service

Add or update these settings in `.env.nonprod`. Keep your real judge key local. It is separate from the provider key entered in AI connection.

```
REMYMY_EVAL_ENABLED=true
REMYMY_EVAL_SAMPLE_RATE=1
REMYMY_EVAL_K=6
REMYMY_EVALUATION_DB=/app/target/evaluations.sqlite3
JEV_API_KEY=YOUR_TYPESAFE_JEV_KEY
```

`REMYMY_EVAL_SAMPLE_RATE=1` records every eligible recommendation; reduce it for sampling. Setting capture does not itself run the judge. The optional `REMYMY_EVALUATION_TOKEN` is for protected human-review endpoints, not the judge API key.

Step 21 - Apply settings and generate a recommendation

```
docker compose --env-file .env.nonprod --profile recommender \
  up -d mcp
```

Use Discover to request actual meal recommendations as on page 9. A greeting or connection test does not create a meal-ranking snapshot. The current implementation records the ranked candidate pool and selected top IDs; it does not parse a separate final three-day plan from the model's prose.

Step 22 - Confirm a snapshot exists

```
docker compose --env-file .env.nonprod exec -T mcp python - <<'PY'
import os, sqlite3
path = os.environ['REMYMY_EVALUATION_DB']
db = sqlite3.connect('file:' + path + '?mode=ro', uri=True)
rows = db.execute(
    'SELECT id, created_at FROM ranking_evaluations '
    'ORDER BY created_at DESC LIMIT 5'
).fetchall()
for row in rows:
    print(*row)
print('Recent snapshots:', len(rows))
db.close()
PY
```

Expect at least one row after a successful recommendation. If the database/table is missing or there are no rows, confirm capture is enabled inside the recreated MCP container, that the full-stack profile is running and that the request returned actual candidates. Existing snapshots remain on the MCP-data volume between restarts.

10. Run the judge and export its report

Step 23 - Grade one captured recommendation first

```
docker compose --env-file .env.nonprod exec -T mcp \
  python -m eval.judge_eval --limit 1
docker compose --env-file .env.nonprod exec -T mcp \
  cat target/eval/jev_report.json
```

This selects the latest stored snapshot, hydrates its Food.com recipes, grades the candidate pool using TypeSafe Jev and computes metrics. It sends recipe/request content to the judge service and may incur usage charges. `--limit 1` limits snapshots, not the number of candidate-level judge requests. Successful judgments are cached in the evaluation database for reuse.

To process more snapshots, increase `--limit`, for example to 25. To compute the report without posting scores to Langfuse, add `--no-push`; this still calls Jev for uncached judgments and requires the key.

Step 24 - Copy the report into Reports

From `SystemCode/remymy`, the repository's Reports folder is two levels up:

```
docker compose --env-file .env.nonprod cp \
  mcp:/app/target/eval/jev_report.json \
  ../../Reports/jev_report.json
```

Step 25 - Check that the report contains evidence

Field	What to check
`plans`	Positive count of snapshots actually graded
`averaged_plans`	Number included in aggregate ranking averages
`averages`	Ranking metrics; null means undefined or no eligible evidence
`pool_lt_k`	Snapshots with fewer candidates than K; excluded from aggregate averages
`per_plan`	Candidate count, trace ID, output/constraint metrics and content/popularity baselines
`scores_pushed_to_langfuse`	Scores posted; can be zero with `--no-push` or absent trace IDs

Compare ranker metrics against the baselines on the same captured pool. A small sample is a smoke check, not proof of general recommendation quality. Keep the input settings, judge version, data/model version and evaluation date with any reported result.

Current limitations: output metrics describe selected top recommendations. The independent violation check is a partial ingredient screen; it does not check every allergy or budget condition. The judge hydrator currently omits `minutes`, so a zero violation rate does not validate cooking-time compliance. Review the actual recipes and requests as well.

11. Inspect traces and scores in Langfuse

Step 26 - Open the local project

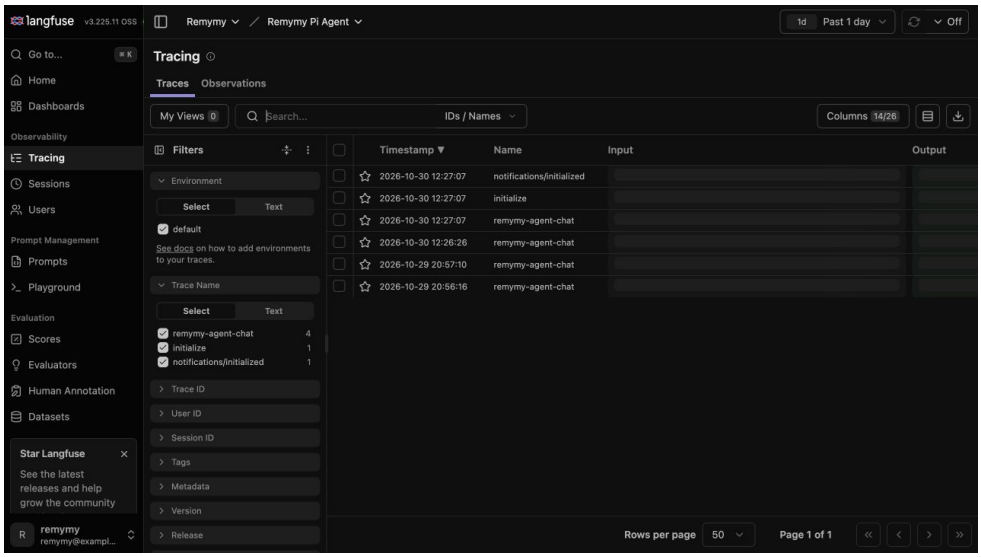
1. Visit `http://localhost:3030`. The loopback-only gateway uses the pre-seeded local development session.

2. Open **Remymy Pi Agent** using **Go to project** if you land on Organizations.

3. Choose **Tracing** in the sidebar and adjust the date range to include your test.

4. Search for `remymy-agent-chat`. Open a trace and inspect input, output, model observations and nested `mcp.client.call` observations.

5. Check the MCP arguments, returned content and errors. `remymy-mcp-probe` traces represent standalone MCP checks.



Langfuse tracing view

Step 27 - Find the evaluation results

After the judge runs with score posting enabled, open **Scores** under Evaluation or inspect scores on the corresponding trace. Match the `trace_id` in the JSON report to the trace. Ranker scores use `jev-judge-pool`; selected-output and constraint scores use `jev-judge-output` as their source metadata. Allow time for ingestion, then refresh.

Metric	Reading it
`nDCG@6`	Higher is better; 1 is ideal ordering by judge grades, not a probability
`Precision@6`, `Recall@6`	Share good in top results; share of good pool candidates retrieved
`pairwise_accuracy`	Share of unequal-grade pairs ordered correctly
`plan_mean_grade`, `plan_share_good`	Mean selected grade (0-3) and share graded at least 2
`plan_repeat_rate`, `diversity`, `coverage`	Repeats, ingredient dissimilarity, and selected share of that pool
`violation_rate`, `over_filtering_rate`	Detected selected-item violations; fewer items than requested

No scores after a chat is expected until an evaluation posts them. The app's automatic JEV metric map is empty in this version; the explicit Python judge command is the supported scoring route. Langfuse's Evaluators page does not launch this repository's Python runner automatically.

12. Check other evaluation paths

Behavior metrics from impressions and later saves

When Discover displays recommendations, the browser logs the ranked impression. The behavior runner checks whether that same guest later saved an item in the top K. A like/dislike is not a save. The saved record's time must be at or after the impression.

```
docker compose --env-file .env.nonprod exec -T api \
  python -m eval.behavior_metrics --k 6 --no-push \
  --out /state/behavior_report.json
docker compose --env-file .env.nonprod cp \
  api:/state/behavior_report.json ../../Reports/behavior_report.json
```

Check `event_count`, `HR@6`, `MRR@6` and `skipped_trace_id_count`. HR is the share of impression events with a later save in the top K. MRR rewards saves at higher ranks. With no events, metrics are null; events without a later qualifying save count as zero. Remove `--no-push` to post scores to matching Langfuse traces using the configured API credentials.

Because the chat popup has no save control, a qualifying save currently needs a matching stored recipe via Meals or the API. Saving an unrelated browsed recipe does not make the impression a hit. Food.com IDs are normalized to stored `openrecipes:` IDs for this comparison, but the stored record must actually exist.

Optional - Reproduce the offline hybrid benchmark

This evaluates the trained collaborative model against historical Food.com interactions. It is separate from live chat judging and browser behavior. You need `RAW_interactions.csv`, `PP_recipes.csv`, the trained NPZ artifact and processed splits under `data`.

```
docker compose --env-file .env.nonprod exec -T models \
  python prepare_cf_splits.py --data-dir /data/raw/food-com \
  --output-dir /tmp/foodcom-cf
docker compose --env-file .env.nonprod exec -T models \
  python evaluate_hybrid.py --data-dir /tmp/foodcom-cf \
  --raw-dir /data/raw/food-com \
  --artifact /data/models/hybrid-foodcom-bpr.npz \
  --samples 6000 --output /tmp/hybrid-evaluation.json
docker compose --env-file .env.nonprod cp \
  models:/tmp/hybrid-evaluation.json ../../Reports/hybrid-evaluation.json
```

The container's `/data` mount is read-only, so these commands write temporary outputs to `/tmp` and copy the final report out. Inspect HR@10, MRR@10 and nDCG@10 across model variants. These are sampled-negative, warm-item validation metrics, not a full-catalog production measurement. See `models/README.md` for training and experiment details.

13. Troubleshooting

Use the relevant check below, correct the underlying setting or asset, then rerun the affected startup or evaluation step.

Symptom	What to do
Cannot connect to Docker daemon	Launch Docker Desktop, wait for the engine, then rerun `docker info`.
A port is already allocated	Change the relevant port in `.env.nonprod`, then rerun Compose. Alternatively use the helper and read its chosen URLs.
`api-migrate` exits nonzero	Read its logs. Supply a valid prepared database or both source paths under `/data`. Do not delete volumes to fix a missing source.
Models or MCP absent from `ps`	Use `--profile recommender` and check both the raw catalog file and matching NPZ artifact.
Meals empty or unavailable	Check API `/ready`, the migration logs and `/api/recipes?limit=2`. A provider key is not required for this screen.
API recommendations return 503	Wait for model loading, inspect API logs, and check the priced catalog. `price_mapped_nutrients.csv` must be a file with data, not a directory.
Similar recipes unavailable	Provide the clustering joblib artifact and rebuild the API image; basic browsing still works without it.
Tag suggestions unavailable	Check classifier weights and the API model download/loading logs. Pick tags manually to continue.
Discover says Not connected	Open AI connection, enter a valid provider key and test the selected model. Check provider/network errors and session expiry.
Connected chat has no meals	Check MCP/model health and dietary filters. Relax optional preferences deliberately and retry.
Judge says JEV_API_KEY is not set	Set it in `.env.nonprod` and recreate `mcp` with `up -d mcp`. A container restart alone does not apply changed environment settings.
Judge report has zero plans	Enable capture, apply settings, then generate a successful meal recommendation. A greeting or empty result has no ranking evidence.
Scores missing in Langfuse	Check `scores_pushed_to_langfuse`, matching trace IDs, project/date filters and Langfuse health. `--no-push` intentionally skips score posting.
Behavior HR/MRR are zero or null	Inspect the event count and later saves. Likes do not qualify; zero and missing evidence mean different things.

Collect focused diagnostics

```
docker compose --env-file .env.nonprod --profile recommender ps --all
docker compose --env-file .env.nonprod logs --tail 80 api-migrate api
docker compose --env-file .env.nonprod logs --tail 80 mcp models
docker compose --env-file .env.nonprod logs --tail 80 \
  langfuse-web langfuse-worker langfuse-gateway
```

The stack also writes hourly UTC log files under `logs/<service>/`. Default retention is 168 hours. Share only the relevant diagnostic excerpt after removing keys and private request content.

14. Stop, restart and finish the walkthrough

Stop the stack while keeping data

```
docker compose --env-file .env.nonprod --profile recommender down
```

This removes containers and the Compose network while retaining named volumes. Avoid `down -v` unless you intentionally want to erase the persistent application, evaluation and Langfuse data. With the startup helper, Ctrl+C performs its normal shutdown.

Start again with existing images and data

```
docker compose --env-file .env.nonprod --profile recommender \
  up -d --wait --wait-timeout 900
```

Use `up --build` after changing application code or image inputs. After an environment edit, use `up -d` for the affected service so Compose can recreate it. Reuse the same browser to recover the same guest's records.

Completion checklist

- Docker answers `docker info` and long-running services are healthy.
- Migration completed with exit code 0; `/ready` is OK and Meals has records.
- Preferences are saved and at least a few distinct ratings appear in History.
- A bookmarked recipe appears in Saved, and an added recipe appears in My recipes.
- AI connection succeeds and Discover returns a real MCP-backed meal suggestion.
- Capture is enabled, a ranking snapshot exists, and the judge report has `plans > 0`.
- The JSON report is copied into Reports and its trace ID matches the Langfuse trace.
- Judge scores are present when posting was enabled; behavior results are read with their event count.